Weapon Detection Systems for Public Security: A YOLO26n Approach in Resource-Constrained Environments

Pau [redacted] *

[redacted]
Universitat Politècnica de Catalunya (UPC), [redacted], Spain

Abstract

This project addresses the increasing need for automated security asolutions by developing a robust weapon detection system. Utilizing YOLO26n architecture, this research focuses on achhieving an optimal balance between detection accuracy and computational performance, customized for resource-constrained enviroments. The methodology involves a comprehensive pipeline, from the Dataset selection to the model and finally the Bayesian optimization via Obtuna for hyperparameter tunning. Results demonstrate that while the default YOLO configuration with MuSGD optimizer reaches slightly higher final peak metrics over 100 epochs, the Optuna-tuned model provides a noticeably more stable convergence path and reaches its performance plateau within the first 40 epochs. This suggests that automated tuning can be used to reduce training time and energy consumption while maintaining the high reliability required for real-time security monitoring in resource-limited scenarios.

Keywords— *Computer Vision, Object Detection, YOLO26n, Bayesian Optimization, Public Security*

Introduction

Mass shootings have been increasing in the last years wether it is related to gang activity, school shootings or terrorism acts. For example, the Regional Gun Violence Research Consortium at the Rockefeller Institute of Government in New York [1], only tracks shootings that occur in public or populated places, involving at least two victims (injured or killed), and excludes incidents related to gang violence or terrorist activity. By its definition, there have been 12 mass shootings in 2025. Meanwhile, the Gun Violence Archive which counts all instances in which four or more people were shot (injuries and deaths), not including the shooter, and regardless of location, reported over 300 mass shootings in 2025 [2].

This project proposes an automated surveillance framework designed to identify weapon-related threats within critical infrastructure via existing CCTV networks. By integrating high-speed object detection with resource efficient architectures, the system aims to provide actionable intellifence in real-time, reducing response times in high-stakes security scenarios.

*Corresponding author: [redacted].
Submitted: March 12, 2026, **Published:** —

Materials and Methods

Dataset

The development of a reliable weapon detection system for public security requires a diverse and high-quality dataset that simulates real-world surveillance conditions. For this project, the **weapon-detection-uu1ng-rozbs** [3] dataset from RoboFlow Universe was selected, providing an extensiv compilation of 9.9k samples. This choice is justified by the dataset's inclusion of images from both CCTV and standard cameras, which is essential for training a model capable of operating in varied public enviroments. No preprocessing steps nor augmentations were applied.

The dataset distribution across the primary classes is detailed in Table 1. The remaining percentage of the total 9.9k samples corresponds to "null" or background images.

Table 1: *Distribution of the dataset classes.*

Dataset Split	Images	(%)
Train	6,995	69.99%*
Valid	2,000	20%*
Test	999	9%*
Total	9,994	98,99%*

**Rounded*

Model Selection and Architecture

In computer vision tasks there are plenty of architectures for object detection tasks. Key ones include **YOLO** (You Only Look Once) [4], **CNN** (Convolutional Neural Networks) [5], and **SSD** (Single Shot Multibox Detector) [6].

CNNs are specialized multileyer neural networks designed for visual tasks. They use convolutional filters to extract meaningful spatial features and pooling layers to reduce the feature-map size while preserving important information. Their structure enables progressive learning from basic edges and shapes to complext object representations in deeper layers, making them effective for diverse computer vision tasks.

Although CNNs have been the preferred method for computer vision tasks in the past, they have become the backbone for more powerful and specialized methods.

R-CNNs (Region-based Convolutional Neural Network) for example, are also among many pioneering approaches of applying deep learning to object detection [7]. It first extracts many (e.g., 2000) region proposals from the input image (e.g., anchor boxes can also be considered as region proposals), labeling their classes and bounding boxes (e.g., offsets). Later on, improved versions of R-CNN arised such as **Fast R-CNNs** and **Faster R-CNN**. The first one leverages functional maps for candidate regions, while R-CNN relies on pixel data from local detection areas. The second one, integrates proposal generation into the network. Comparison between the three of them in Table 2.

Table 2: *Speed comparison of R-CNN variants. Reproduced from [8].*

Model	Time	Main Feature
R-CNN	$\sim$ 47s	Separate CNN/region
Fast R-CNN	$\sim$ 2s	Shared features
Faster R-CNN	$\sim$ 0.2s	Integrated RPN

SSD uses a single deep neural network. This approach, discretizes the output space of bounding boxes into a set of default boxes over different aspect ratios and scales per feature map location.

YOLO is another approach [9]. This algorithm requires only one forward propagation pass through the neural network to make predictions, unlike previous region proposal-based techniques like R-CNN that require thousands for a single image. By using multidimensional separation and class probabilities, it handles the detection job as a regression issue rather than a classification one. The input image is represented as a grid of tensors using a "CNN". The technique "predicts bounding boxes for objects" and the associated class probabilities for each grid cell. See **Figure 1** for more details.

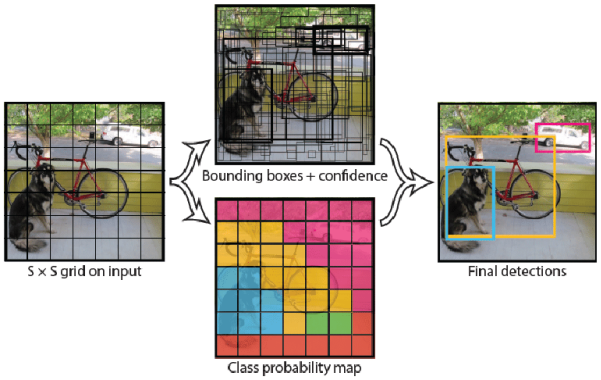

Figure 1: ***YOLO Detection Formulation.*** *The predictions are encoded as an $S \times S \times (B \cdot 5 + C)$ tensor. Reproduced from [4].*

One advantage of this algorithm is that it can extract detection regions without the need for a separate network, which contributes to its enhanced processing speed and overall performance.

Ultralytics YOLO [10], an open-source model implementation of YOLO from the **Ultralytics** company, offers different type of models with different parameters (N, S, M, L, X). These, are pre-trained on the **COCO Dataset** [11], which offers large-scale object detection, segmentation, and captioning dataset with 330K images (>200K labeled), 1.5 million object instances, 80 object categories, 91 stuff categories, 5 captions per image and 250,000 people with keypoints.

Currently, **YOLO26** represents the most advanced model developed by the Ultralytics team. A detailed comparative analysis of the available configurations for this model is provided in Table 3, while Figure 2 illustrates the performance trade-offs regarding mAP_{50-95}.

Table 3: *Comparison of YOLO26 models by mAP, parameters, and FLOPs.*

Model	mAP_val 50—95	Params (M)	FLOPs (B)
YOLO26n	40.9	2.4	5.4
YOLO26s	48.6	9.5	20.7
YOLO26m	53.1	20.4	68.2
YOLO26l	55.0	24.8	86.4
YOLO26x	57.5	55.7	193.9

The **YOLO26n** model was selected for this implementation due to its superior efficiency-to-precision ratio. For systems requiring autonomous monitoring

via CCTV, the Nano architecture provides the necessary throughput for real-time processing. Furthermore, the selection of a lightweight model ensures the system remains functional on low-specification hardware, facilitating broader deployment in scenarios where high-performance computing infrastructure is unavailable or expensive.

The implementation details, including the training scripts and the deployment dashboard, are hosted on GitHub [12].

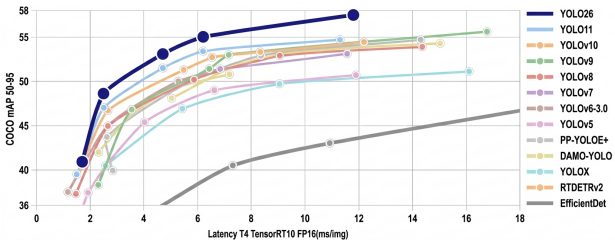

Figure 2: *COCO mAP 50:95—latency comparison of YOLO and real-time object detectors on the T4 (TensorRT FP16) [13]. (*) Image enhanced with AI.*

Results and Discussion

Hyperparameter Optimization (Bayesian Search)

To maximize the detection capabilities of the YOLO26n architecture, a hyperparparameter tuning phase was conducted using **Optuna** [14]. This phase utilized a Bayesian optimization strategy with a *MediaPruner*— a pruner using the median stopping—rule to effectively navigate the seatch space over 30 trials.

The objective function was designed to maximize the mAP_{50-95}, evaluating key parameters including learning rate, momentum and loss gain coefficients (box, cls and dfl).

Table 4: *Best Hyperparameters Found by Optuna*

Parameter	Value
Learning Rate (lr_0)	0.000647
Weight Decay	7.127×10^{-5}
Momentum	0.8576
Warmup Epochs	2.853
Box Loss Gain	7.635
Class Loss Gain	0.298
DFL Loss Gain	1.735

Phase I: Short-term Convergence Comparison (30 Epochs)

In the first phase, we compared the besilne model (default parameters) against the tuned model over a short 30 epoch cycle to observe early convergence behaviour.

As shown in the figure 3, the **Tuned** model (blue line) demonstrates a substantially more stable and faster learning curve during the initial 10 epochs, particularly in mAP_{50-95} and **Recall**. While the baseline eventually catches up in raw metrics by epoch 30, the tuned model exhibits fewer fluctuations, suggesting a stronger optimization path.

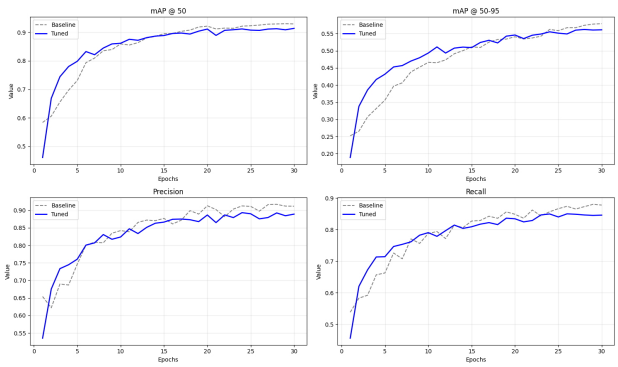

Figure 3: *Phase I Results Summary.*

Phase II: Long-term Training and Stability (100 Epochs)

To validate the effectiveness of the optimized parameters in a full production-ready training cycle, both the baseline and the Tuned models were trained for a duration of 100 epochs. During this extended training period, several key observations regarding model convergence and stability were made:

- **Metric Stability:** The Tuned model exhibits notably stable mAP curves throughout the entire 100-epoch duration. While the baseline model shows a performance fluctuation between epochs 85 and 95, the Tuned model maintains a consistent and reliable trajectory. This stability suggests that the hyperparameters found via Optuna not only optimize accuracy but also guarantees a more predictable convergence path.
- **Generalization:** During the initial 30 to 40 epochs, the Tuned model demonstrates superior performance in both Precision and Recall. This indicates that the Bayesian-optimized parameters allow the model to adapt to the dataset's specific features considerably faster than the baseline, reaching a high-performance plateu earlier in the training process.

- **Final Metrics:** After the first 40 epochs, the baseline model eventually stabilizes at a slightly higher performance level than the Tuned model in most categories. However, both models converge to a nearly identical Recall (0.8624 for Tuned vs. 0.8623 for Baseline).

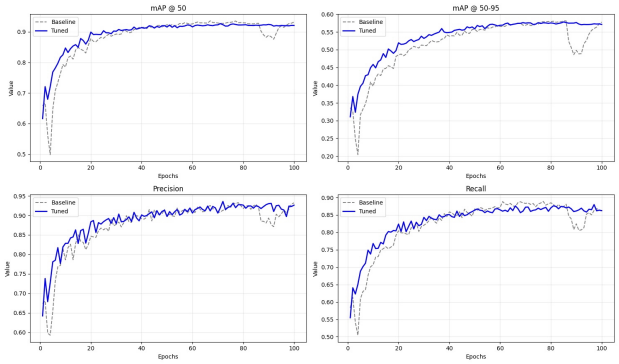

Figure 4: *Phase II Results Summary.*

The numerical summary of the final epoch metrics is presented in Table 5.

Table 5: *Phase II Results Summary (100 Epochs)*

Metric	Baseline	Tuned	Improvement
mAP @ 50	0.9318	0.9207	-1.20%
mAP @ 50-95	0.5779	0.5712	-1.16%
Precision	0.9322	0.9263	-0.63%
Recall	0.8623	0.8624	+0.01%

Discussion

The experimental results show a trade-off between early convergence and absolute final performance. The default YOLO configuration, utilizing its native **MuSGD** optimizer (a hybrid of SGD [15] and Muo [16]), proves to be exceptionally well-calibrated for long-term adaptation, eventually reclaiming a slight edge over the Tuned model. This suggests that the default architectural priors of YOLO are robust enough to extract peak performance when given sufficient training time.

However, the Tuned model's performance remains the primary focus for reliable deployment. By providing a smoother convergence without the late-stage fluctuations seen in the baseline, the Optuna-optimized model presents itself as a more dependable choice for environments requiring consistent behavior. The fact that the Tuned model was able to reach its peak performance much faster (within the first 40 epochs) also suggests that hyperparameter tuning could be used to reduce total training time and energy consumption without a major loss in detection capabilities.

References

[1] *Mass shooting factsheet*. 2026. URL: https://rockinst.org/gun-violence/mass-shooting-factsheet/.

[2] URL: https://repository.gheli.harvard.edu/repository/collection/resource-pack-gun-violence-and-public-health/resource/12364/.

[3] Justin Treen. *Weapon detection Dataset*. https://universe.roboflow.com/justin-treen-0ewy1/weapon-detection-uu1ng. Open Source Dataset. Feb. 2024. URL: https://universe.roboflow.com/justin-treen-0ewy1/weapon-detection-uu1ng.

[4] Joseph Redmon et al. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. arXiv: 1506.02640 [cs.CV]. URL: https://arxiv.org/abs/1506.02640.

[5] Zhong-Qiu Zhao et al. "Object detection with deep learning: A review". In: *IEEE transactions on neural networks and learning systems* 30.11 (2019), pp. 3212–3232.

[6] Wei Liu et al. "Ssd: Single shot multibox detector". In: *European conference on computer vision*. Springer. 2016, pp. 21–37.

[7] Ross Girshick et al. "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2014.

[8] Ghansham More et al. "Comparison of object detection algorithms CNN, YOLO and SSD". In: *International Journal of Scientific Research and Technology* (2024).

[9] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. 2020. arXiv: 2004.10934 [cs.CV]. URL: https://arxiv.org/abs/2004.10934.

[10] Ultralytics. *Ultralytics YOLO Docs*. https://docs.ultralytics.com/. 2024.

[11] Tsung-Yi Lin et al. *Microsoft COCO: Common Objects in Context*. 2015. arXiv: 1405.0312 [cs.CV]. URL: https://arxiv.org/abs/1405.0312.

[12] Pau Camós. *Weapon Detection YOLO26n Implementation*. 2026. URL: https://github.com/cyberpabs/Weapon-Detection-YOLO26n.

[13] Ultralytics. *Ultralytics Benchmark Mode Documentation*. https://docs.ultralytics.com/modes/benchmark/. 2024.

[14] Takuya Akiba et al. *Optuna: A Next-generation Hyperparameter Optimization Framework*. 2019. arXiv: 1907.10902 [cs.LG]. URL: https://arxiv.org/abs/1907.10902.

[15] Azar Louzi. *Stochastic Gradient Descent Revisited*. 2025. arXiv: 2412.06070 [math.OC]. URL: https://arxiv.org/abs/2412.06070.

[16] Keller Jordan et al. *Muon: An optimizer for hidden layers in neural networks*. 2024. URL: https://kellerjordan.github.io/posts/muon/.